

Support Apprenant

Module 1 — Les fondamentaux de Python

- Comprendre la philosophie et les cas d'usage de Python
- Installer et configurer un environnement de développement Python
- Maîtriser la syntaxe de base : variables, types, opérateurs
- Utiliser les structures de contrôle : conditions, boucles, gestion d'erreurs

Environnement & premiers pas

QU'EST-CE QUE PYTHON ?

- Langage interprété, haut niveau, généraliste
- Créé par Guido van Rossum en 1991
- Philosophie : "Readable code is better than clever code" (PEP 20)
- Versions actives recommandées : Python 3.12 / 3.13 (3.10 et 3.11 en fin de vie en 2026)
- Écosystème gigantesque : +500 000 packages sur PyPI

À retenir :

Le Zen de Python (PEP 20) résume la philosophie en 19 aphorismes, dont les plus connus : "Simple is better than complex", "Readability counts", "There should be one obvious way to do it".

OÙ UTILISE-T-ON PYTHON ?

- Data science & IA : NumPy, Pandas, scikit-learn, TensorFlow, PyTorch
- Développement web : Django, Flask, FastAPI
- Scripting & automatisation : scripts système, CI/CD, DevOps
- Finance & sciences : calcul numérique, simulations
- Cybersécurité : outils d'audit, pentesting

POURQUOI DES ENVIRONNEMENTS VIRTUELS ?

++++

graph TD; SYS["Python système(global)"]

```

subgraph SANS["❌ Sans environnement virtuel"]
    P1A["Projet A\nrequests 2.28"]
    P2A["Projet B\nrequests 2.31"]
    CONF["💥 Conflit de versions !"]
    P1A -- installe --> CONF
    P2A -- écrase --> CONF
end

subgraph AVEC["✅ Avec environnements virtuels"]
    V1["venv Projet A\nrequests 2.28\nflask 2.3"]
    V2["venv Projet B\nrequests 2.31\ndjango 4.2"]
    V3["venv Projet C\nnumpy 1.24\npandas 2.0"]
end

SYS --> V1
SYS --> V2
SYS --> V3

```

++++

INSTALLATION DE L'ENVIRONNEMENT

- **Python** : télécharger depuis python.org (cocher "Add to PATH" sur Windows)
- **Vérification** : `python --version` ou `python3 --version`
- **pip** : gestionnaire de paquets, inclus avec Python 3.4+
- **IDE recommandés** : VS Code (extension Python/Pylance) ou PyCharm

```

# Vérifier l'installation

python --version      # Python 3.12.x
pip --version         # pip 24.x

# Créer un environnement virtuel
python -m venv mon_env

# Activer l'environnement
source mon_env/bin/activate  # Linux/Mac
mon_env\Scripts\activate    # Windows

```

À retenir :

Un environnement virtuel isole les dépendances d'un projet. Sans cela, tous vos projets partagent les mêmes versions de bibliothèques, ce qui crée des conflits. Toujours créer un venv par projet !

PREMIER SCRIPT PYTHON

```
# hello.py – Mon premier script Python

# Afficher un message
print("Bonjour, Python !")

# Demander une saisie
prenom = input("Votre prénom : ")
print(f"Bonjour, {prenom} !")

# Calcul simple
annee_naissance = int(input("Année de naissance : "))
age = 2024 - annee_naissance
print(f"Vous avez environ {age} ans.")
```

- Exécution : `python hello.py`
- Interpréteur interactif : taper `python` ou `python3`
- REPL (Read-Eval-Print Loop) : idéal pour tester rapidement

SYNTAXE DE BASE & PEP 8

- **Indentation** : 4 espaces (pas de tabulations) — obligatoire en Python
- **Commentaires** : `#` en ligne, `"""docstring"""` pour documenter
- **Nommage** : `snake_case` pour variables/fonctions, `PascalCase` pour classes
- **Longueur de ligne** : PEP 8 recommande 79 caractères ; la plupart des projets modernes utilisent 88 (black/ruff) ou 99 caractères
- **Pas de point-virgule** en fin de ligne

```
# Bon style Python (PEP 8)
nombre_apprenants = 12
taux_reussite = 0.85

def calculer_moyenne(notes):
    """Calcule la moyenne d'une liste de notes."""
    return sum(notes) / len(notes)

# Mauvais style (à éviter)
x=12; y=0.85
def CalcMoy(n): return sum(n)/len(n)
```

À retenir :

**** Qu'est-ce que PEP 8 ?

Une PEP (Python Enhancement Proposal) est un document officiel qui propose des améliorations au langage Python. La PEP 8 — publiée en 2001 — est le guide de style de référence : elle définit les conventions d'écriture pour rendre le code Python lisible et cohérent entre développeurs.

Outils associés :

- `black` — formate automatiquement votre code selon PEP 8
- `flake8` / `pylint` — détectent les violations de style
- `isort` — trie les imports automatiquement ****

Types & variables

TYPES PRIMITIFS EN PYTHON

- `int` : entiers (taille illimitée en Python 3)
- `float` : nombres décimaux (IEEE 754 double précision)
- `str` : chaînes de caractères (immuables, Unicode)
- `bool` : `True` ou `False` (sous-classe de `int`)
- `None` : valeur nulle (équivalent de `null` en JavaScript/Java ou `nil` en Ruby/Kotlin)

```
age = 30          # int
prix = 19.99      # float
nom = "Alice"     # str
actif = True      # bool
valeur = None     # NoneType

# Vérifier le type
print(type(age))  # <class 'int'>
print(type(nom))  # <class 'str'>
print(isinstance(age, int)) # True
```

VARIABLES & AFFECTATION

- Pas de déclaration de type obligatoire — Python l'infère
- Une variable est un **nom** qui pointe vers un **objet** (→ slide suivante)
- Affectation multiple et déstructuration possibles
- **Type hints** (Python 3.5+) : optionnels mais recommandés

```

# Affectation simple
x = 10
x = "maintenant une chaîne" # OK, typage dynamique

# Affectations multiples
a = b = c = 0
x, y, z = 1, 2, 3

# Déstructuration
prenom, nom = "Alice", "Dupont"
premier, *reste = [1, 2, 3, 4, 5]
# premier = 1, reste = [2, 3, 4, 5]

# Type hints (annotations, non contraignantes)
score: int = 100
message: str = "Bonjour"

```

À retenir :

Les type hints sont de plus en plus utilisés en Python professionnel. Ils servent à la documentation, à la détection d'erreurs avec *mypy*, et à l'autocomplétion dans les IDE.

VARIABLES — MODÈLE OBJET DE PYTHON

++++

Espace des noms (variables)

Objets en mémoire (heap)

1 x = 42

variable x

int 42

2 y = x

variable x

variable y

int 42

même objet ! `id(x) == id(y)`

3 x = "hello"

variable x

variable y

str "hello"

int 42

x change de cible y pointe encore sur 42

💡 Variable Python = étiquette collée sur un objet — pas une boîte qui le contient ++++

OPÉRATEURS

- Arithmétiques : `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Comparaison : `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logiques : `and`, `or`, `not`

```
print(10 // 3)    # 3    - division entière
```

```
print(10 % 3)     # 1    - modulo (reste)
```

```
print(2 ** 8)     # 256  - puissance
```

```
# Exemple concret : vérifier qu'un âge est valide
```

```
age = 17
```

```
majeur = age >= 18
```

```
valide = age > 0 and age < 120
```

```
print(majeur, valide) # False True
```

CHAÎNES DE CARACTÈRES

- Guillemets simples ou doubles — équivalents

- f-strings (Python 3.6+) : méthode recommandée pour formater
- Méthodes utiles : `upper()`, `lower()`, `strip()`, `split()`, `replace()`, `startswith()`
- Chaînes immuables : toute modification crée un nouvel objet (→ slide suivante)

```
nom = "Alice"
ville = "Paris"

# F-string (recommandé)
message = f"Bonjour {nom}, vous êtes à {ville.upper()}"

# Méthodes utiles
texte = "  Bonjour le monde  "
print(texte.strip())      # "Bonjour le monde"
print(texte.strip().split()) # ['Bonjour', 'le', 'monde']

mots = ["Python", "est", "génial"]
print(" ".join(mots))     # "Python est génial"

# Slicing de chaînes
s = "Python"
print(s[0])              # 'P'
print(s[-1])             # 'n'
print(s[1:4])            # 'yth'
print(s[::-1])           # 'nohtyP' (inversé)
```

À retenir :

Avant les f-strings, on utilisait `"Bonjour %s" % nom` ou `"Bonjour {}".format(nom)`. Vous verrez ces syntaxes dans du code existant, mais préférez les f-strings pour le nouveau code.

CHAÎNES IMMUABLES — ILLUSTRATION

++++



Espace des noms

Heap (mémoire)

① `s = "hello"`

`s`

`str "hello"`

② `s.upper()`

`str (nouveau) "HELLO"`

retourne un nouvel objet `s` inchangé ✓

③ `s = s.upper()`

`s`

plus référencé

💡 Toute méthode de chaîne renvoie un nouvel objet — elle ne modifie jamais l'original

++++

CONVERSION DE TYPES

- `int()`, `float()`, `str()`, `bool()` : conversions explicites
- Conversions implicites : Python n'en fait presque pas (contrairement à JS)

- Valeurs “par défaut” : `0`, `0.0`, `""`, `[]`, `{}`, `None`, `False`

```
# Conversions explicites
age_texte = "25"
age = int(age_texte)      # 25
prix = float("19.99")     # 19.99
texte = str(42)           # "42"

# Pièges courants
print(int("3.14"))        # ValueError ! Passer par float d'abord
print(int(float("3.14"))) # 3 ✓

# Valeurs par défaut
valeurs = [0, "", [], None, False, 0.0]
for v in valeurs:
    print(f"{repr(v)} → {bool(v)}") # Toutes → False
```

Structures de contrôle

CONDITIONS : IF / ELIF / ELSE

- Pas de parenthèses obligatoires autour de la condition
- Les blocs sont délimités par l'indentation (4 espaces)
- `elif` remplace le `else if` d'autres langages
- Expression ternaire : `valeur_si_vrai if condition else valeur_si_faux`

```
note = 14

if note >= 16:
    mention = "Très bien"
elif note >= 14:
    mention = "Bien"
elif note >= 12:
    mention = "Assez bien"
elif note >= 10:
    mention = "Passable"
else:
    mention = "Insuffisant"

print(f"Note : {note}/20 – {mention}")
```

```
# Expression ternaire
statut = "Admis" if note >= 10 else "Recalé"
print(statut)
```

À retenir :

Python 3.10 introduit le *match/case* (pattern matching), similaire au *switch* d'autres langages. Utile pour de nombreux cas de branchement.

BOUCLES : FOR & WHILE

- `for` : itération sur n'importe quel itérable (liste, chaîne, range, dict...)
- `while` : boucle tant qu'une condition est vraie
- `break` : quitter la boucle immédiatement
- `continue` : passer à l'itération suivante
- `else` sur une boucle : exécuté si la boucle se termine sans `break`

```
# for avec range
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)
```

```
# for avec enumerate
fruits = ["pomme", "banane", "cerise"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

```
# while avec break
compteur = 0
while True:
    compteur += 1
    if compteur >= 5:
        break
print(f"Compteur final : {compteur}")
```

BOUCLES : FOR & WHILE (SUITE)

```
# zip – itérer sur deux listes en parallèle
noms = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]

for nom, score in zip(noms, scores):
    print(f"{nom} : {score}/100")
```

```

# continue – sauter les pairs
for i in range(10):
    if i % 2 == 0:
        continue
    print(i) # Affiche 1, 3, 5, 7, 9

# else sur une boucle
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            break
    else:
        print(f"{n} est premier")

```

À retenir :

La clause `else` sur une boucle `for` ou `while` est une particularité de Python peu connue. Elle s'exécute uniquement si la boucle s'est terminée normalement (sans `break`).

QU'EST-CE QU'UNE EXCEPTION ?

++++

méthode_appelante()

try: result = diviser(10, 0) skip

except ZeroDivisionError as e: print(f"Erreur : {e}") result = 0

✓ Exécution continue

diviser(a, b)

a = 10, b = 0 ← paramètres reçus

if b == 0: # ⚠ détecte l'erreur

raise ZeroDivisionError("Division par zéro") ⚡

① appel →

② ← ValueError ⚡

La méthode appelée détecte l'erreur et lève l'exception — l'appelante la capture et reprend le contrôle +
+++

GESTION DES ERREURS

- `try / except` : capturer et gérer les exceptions
- `else` : exécuté si pas d'exception dans `try`
- `finally` : toujours exécuté (nettoyage, fermeture de ressources)
- `raise` : lever une exception manuellement
- Exceptions courantes : `ValueError` , `TypeError` , `FileNotFoundError` , `KeyError`

```
def diviser(a, b):  
    try:  
        resultat = a / b  
    except ZeroDivisionError:  
        print("Erreur : division par zéro !")  
        return None  
    except TypeError as e:  
        print(f"Type incorrect : {e}")  
        return None  
    else:  
        print(f"Résultat : {resultat}")  
        return resultat  
    finally:  
        print("Calcul terminé.")
```

```
diviser(10, 2) # OK
```

```
diviser(10, 0) # ZeroDivisionError
```

```
diviser(10, "a") # TypeError
```

GESTION DES ERREURS — LEVER DES EXCEPTIONS

```
def valider_age(age):  
    """Valide que l'âge est un entier positif."""  
    if not isinstance(age, int):  
        raise TypeError(f"L'âge doit être un entier, reçu : {type(age)}")  
    if age < 0 or age > 150:  
        raise ValueError(f"Âge invalide : {age}")  
    return age  
  
# Créer ses propres exceptions  
class AgeInvalideError(ValueError):  
    """Exception personnalisée pour un âge invalide."""  
    pass  
  
try:  
    valider_age(-5)  
except ValueError as e:  
    print(f"Valeur incorrecte : {e}")  
except TypeError as e:  
    print(f"Type incorrect : {e}")
```

À retenir :

Il est recommandé de créer des exceptions personnalisées pour les erreurs métier de votre application. Cela rend le code plus lisible et facilite la gestion d'erreurs en amont.

Récapitulatif du Module 1

CE QUE VOUS AVEZ APPRIS

- **Python** : langage interprété, multi-paradigme, syntaxe lisible
- **Environnement** : installation, pip, venv, VS Code
- **Types** : int, float, str, bool, None — typage dynamique et fort
- **Variables** : affectation, déstructuration, type hints
- **Opérateurs** : arithmétiques, comparaison, logiques
- **Contrôle** : if/elif/else, for, while, break, continue
- **Erreurs** : try/except/finally, raise, exceptions personnalisées

Module 2 — Structures de données & fonctions

- Maîtriser les collections Python : listes, tuples, sets, dictionnaires
- Écrire des fonctions robustes et réutilisables
- Utiliser les compréhensions pour un code concis et pythonique
- Lire et écrire des fichiers, gérer les chemins avec pathlib

Collections Python

LISTES — CRÉATION & ACCÈS

- Collection **ordonnée**, **mutable**, peut contenir des types mixtes
- Indexation à partir de 0, indexation négative depuis la fin
- Slicing : `liste[début:fin:pas]`
- Longueur : `len(liste)`

```
# Création
vide = []
nombres = [1, 2, 3, 4, 5]
mixte = [42, "Python", True, 3.14, None]

# Accès par index
print(nombres[0])    # 1    (premier)
print(nombres[-1])   # 5    (dernier)
print(nombres[-2])   # 4    (avant-dernier)

# Slicing
print(nombres[1:4])   # [2, 3, 4]
print(nombres[:3])    # [1, 2, 3]
print(nombres[::2])    # [1, 3, 5] (un sur deux)
print(nombres[::-1])  # [5, 4, 3, 2, 1] (inversée)
```

LISTES — MÉTHODES PRINCIPALES

```
fruits = ["pomme", "banane", "cerise"]

# Ajout
fruits.append("datte")    # Ajoute en fin
fruits.insert(1, "abricot") # Insère à l'index 1
```

```

fruits.extend(["figue", "kiwi"]) # Ajoute plusieurs éléments

# Suppression
fruits.remove("banane")          # Par valeur (1ère occurrence)
dernier = fruits.pop()           # Retire et retourne le dernier
del fruits[0]                    # Suppression par index

# Recherche & tri
print("cerise" in fruits)        # True
print(fruits.index("cerise"))    # Index de la valeur
fruits.sort()                   # Tri en place
fruits.sort(reverse=True)       # Tri décroissant
trie = sorted(fruits)           # Retourne une nouvelle liste triée

```

À retenir :

`sort()` modifie la liste en place et retourne *None*. `sorted()` retourne une nouvelle liste et laisse l'originale intacte. Préférez `sorted()` quand vous ne voulez pas modifier l'original.

TUPLES & SETS

Tuples : séquences immuables, utilisés pour les données fixes **Sets** : collections non ordonnées, sans doublons

```

# Tuples
coordonnees = (48.8566, 2.3522) # Paris
point = 3, 7                    # Parenthèses optionnelles
x, y = coordonnees              # Déstructuration

print(coordonnees[0])          # 48.8566
# coordonnees[0] = 0           # TypeError : immuable !

# Sets
couleurs = {"rouge", "vert", "bleu"}
couleurs.add("jaune")
couleurs.discard("vert")       # Pas d'erreur si absent

# Opérations ensemblistes
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)                   # Union      : {1, 2, 3, 4, 5, 6}
print(a & b)                    # Intersect.: {3, 4}
print(a - b)                   # Différence: {1, 2}

```

DICTIONNAIRES

- Collection de paires clé → valeur
- Clés uniques, immuables (str, int, tuple)
- Ordonnés depuis Python 3.7 (ordre d'insertion)
- Accès en $O(1)$ (table de hachage)

```
personne = {
    "nom": "Alice",
    "age": 30,
    "ville": "Paris"
}

# Accès
print(personne["nom"])          # Alice
print(personne.get("email", "N/A")) # N/A (valeur par défaut)

# Modification
personne["age"] = 31
personne["email"] = "alice@exemple.fr"

# Parcours
for cle in personne:
    print(cle)

for cle, valeur in personne.items():
    print(f"{cle} : {valeur}")

# Suppression
del personne["email"]
age = personne.pop("age")      # Retire et retourne
```

À retenir :

`dict.get(cle, default)` est préférable à `dict[cle]` quand la clé peut être absente. Cela évite un `KeyError`. On peut aussi utiliser `collections.defaultdict` pour les cas plus avancés.

COMPRÉHENSIONS DE LISTES

- Syntaxe concise pour créer des listes à partir d'autres itérables
- Remplace les boucles `for` + `append()` par une expression
- Plus rapide et plus "pythonique"

- Peut inclure un filtre avec `if`

```
# Sans compréhension
carres = []
for n in range(10):
    carres.append(n ** 2)

# Avec compréhension de liste
carres = [n ** 2 for n in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Avec filtre
pairs = [n for n in range(20) if n % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Transformation de chaînes
mots = [" Python ", " est ", " super "]
propres = [m.strip().capitalize() for m in mots]

# Compréhension de dict
notes = {"Alice": 14, "Bob": 12, "Charlie": 17}
admis = {nom: note for nom, note in notes.items() if note >= 10}
```

| Fonctions

DÉFINIR UNE FONCTION

- Mot-clé `def`, nom en `snake_case`
- `return` retourne une valeur (sans `return` → retourne `None`)
- **Portée** : les variables locales n'existent que dans la fonction
- Documenter avec une **docstring** entre `"""`

```
def saluer(prenom):
    """Retourne un message de salutation personnalisé."""
    return f"Bonjour, {prenom} !"

def calculer_imc(poids, taille):
    """
    Calcule l'Indice de Masse Corporelle.

    Args:
```

```

    poids: Poids en kilogrammes
    taille: Taille en mètres

Returns:
    float: La valeur de l'IMC
"""
if taille <= 0:
    raise ValueError("La taille doit être positive")
return poids / (taille ** 2)

print(saluer("Alice"))           # Bonjour, Alice !
print(f"{calculer_imc(70, 1.75):.2f}") # 22.86

```

À retenir :

La convention de docstring la plus répandue est le style Google ou NumPy. Des outils comme *sphinx* peuvent générer automatiquement la documentation à partir des docstrings.

PARAMÈTRES AVANCÉS

- Positionnels : ordre imposé
- Nommés (keyword) : ordre libre avec nom explicite
- Valeurs par défaut : paramètres optionnels
- `*args` : nombre variable d'arguments positionnels
- `**kwargs` : nombre variable d'arguments nommés

```

# Valeurs par défaut
def creer_compte(nom, role="utilisateur", actif=True):
    return {"nom": nom, "role": role, "actif": actif}

creer_compte("Alice")
creer_compte("Bob", role="admin")
creer_compte("Charlie", actif=False)

# *args et **kwargs
def afficher_infos(*args, **kwargs):
    for val in args:
        print(f"- {val}")
    for cle, val in kwargs.items():
        print(f"  {cle}: {val}")

afficher_infos("Python", "Formation", langue="fr", niveau="débutant")

```

FONCTIONS LAMBDA & ORDRE SUPÉRIEUR

- **Lambda** : fonction anonyme en une ligne
- `map(func, iterable)` : applique une fonction à chaque élément
- `filter(func, iterable)` : filtre selon une condition
- `sorted(..., key=func)` : tri personnalisé

```
# Lambda
doubler = lambda x: x * 2
carre = lambda x: x ** 2

# map – transformer
nombres = [1, 2, 3, 4, 5]
doubles = list(map(lambda x: x * 2, nombres))
# [2, 4, 6, 8, 10]

# filter – filtrer
pairs = list(filter(lambda x: x % 2 == 0, nombres))
# [2, 4]

# sorted avec clé personnalisée
etudiants = [
    {"nom": "Charlie", "note": 17},
    {"nom": "Alice", "note": 14},
    {"nom": "Bob", "note": 19},
]
par_note = sorted(etudiants, key=lambda e: e["note"], reverse=True)
```

À retenir :

En Python moderne, on préférera les compréhensions de listes aux `map()` et `filter()` pour leur lisibilité. Mais `sorted()` avec `key=` reste très utile et incontournable.

MODULES & IMPORTS

- Un **module** = un fichier `.py`
- Un **package** = un dossier avec `__init__.py`
- `import module` : importe le module entier
- `from module import nom` : importe un élément spécifique
- `as` : alias pour raccourcir le nom
- Modules de la bibliothèque standard très riches

```

# Import complet
import math
print(math.pi)          # 3.14159...
print(math.sqrt(144))   # 12.0

# Import sélectif
from math import pi, sqrt, ceil
print(ceil(3.2))        # 4

# Alias
import datetime as dt
aujourd_hui = dt.date.today()

# Modules standards utiles
import os, sys, re, json, random, collections, itertools, functools

# Votre propre module
# fichier: utils.py
# import utils
# from utils import ma_fonction

```

Fichiers & entrées/sorties

LECTURE ET ÉCRITURE DE FICHIERS TEXTE

- `open(chemin, mode)` retourne un objet fichier
- Toujours utiliser le gestionnaire de contexte `with` (fermeture automatique)
- Modes : `"r"` (lecture), `"w"` (écriture/création), `"a"` (ajout), `"r+"` (lecture+écriture)
- Spécifier l'encodage : `encoding="utf-8"` (recommandé)

```

# Écriture
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Ligne 1\n")
    f.write("Ligne 2\n")
    f.writelines(["Ligne 3\n", "Ligne 4\n"])

# Lecture complète
with open("notes.txt", "r", encoding="utf-8") as f:
    contenu = f.read()          # Tout le fichier en une chaîne

# Lecture ligne par ligne

```

```
with open("notes.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())

# Lecture en liste
with open("notes.txt", "r", encoding="utf-8") as f:
    lignes = f.readlines() # Liste de chaînes avec \n
```

FICHIERS CSV

- Module `csv` de la bibliothèque standard
- `csv.reader` : lecture ligne par ligne (liste)
- `csv.DictReader` : lecture avec noms de colonnes (dict)
- `csv.writer` / `csv.DictWriter` : écriture

```
import csv

# Écriture CSV
etudiants = [
    ["Alice", 14, "Bien"],
    ["Bob", 17, "Très bien"],
    ["Charlie", 11, "Passable"],
]

with open("etudiants.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Nom", "Note", "Mention"])
    writer.writerows(etudiants)

# Lecture avec DictReader
with open("etudiants.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for ligne in reader:
        print(f"{ligne['Nom']} : {ligne['Note']}/20")
```

À retenir :

Pour des fichiers CSV complexes (gros volumes, types mélangés, nettoyage), la bibliothèque *pandas* (Module 4) est bien plus adaptée que le module `csv` standard.

GESTION DES CHEMINS AVEC PATHLIB

- Module `pathlib` (Python 3.4+) : manipulation orientée objet des chemins

- Remplace avantageusement `os.path`
- Fonctionne sur tous les OS (Windows, Linux, Mac)
- Opérateur `/` pour construire des chemins

```
from pathlib import Path

# Créer un chemin
dossier = Path("data")
fichier = dossier / "rapport.csv" # data/rapport.csv

# Infos sur le chemin
print(fichier.name) # rapport.csv
print(fichier.stem) # rapport
print(fichier.suffix) # .csv
print(fichier.parent) # data

# Opérations
dossier.mkdir(exist_ok=True) # Créer le dossier
print(fichier.exists()) # True/False
fichier.unlink(missing_ok=True) # Supprimer

# Lister les fichiers
for f in Path(".").glob("*.py"):
    print(f)

# Lire/écrire directement
contenu = fichier.read_text(encoding="utf-8")
fichier.write_text("nouveau contenu", encoding="utf-8")
```

À retenir :

`Path.home()` retourne le répertoire personnel de l'utilisateur. `Path.cwd()` retourne le répertoire courant. Ces deux méthodes sont très utiles pour des scripts portables.

Récapitulatif du Module 2

CE QUE VOUS AVEZ APPRIS

- **Listes** : collection ordonnée et mutable, slicing, méthodes
- **Tuples** : immuables, idéaux pour les données fixes
- **Sets** : sans doublons, opérations ensemblistes

- Dictionnaires : clé/valeur, accès rapide, parcours
- Compréhensions : syntaxe concise et pythonique
- Fonctions : def, return, paramètres, *args, **kwargs, lambda
- Fichiers : open/with, csv, pathlib

Module 3 — Programmation Orientée Objet

- Comprendre les concepts fondamentaux de l'OOP : classe, objet, encapsulation
- Créer des classes Python avec attributs, méthodes et constructeur
- Utiliser l'héritage, le polymorphisme et les méthodes spéciales
- Appliquer les bonnes pratiques OOP avec dataclasses et organisation en packages

Concepts OOP

POURQUOI LA PROGRAMMATION ORIENTÉE OBJET ?

- **Modélisation** : représenter des entités du monde réel (Utilisateur, Commande, Produit)
- **Encapsulation** : regrouper données et comportements dans un même objet
- **Réutilisabilité** : hériter et étendre sans réécrire
- **Maintenabilité** : code organisé, responsabilités claires
- Python supporte la POO depuis sa création, mais n'y oblige pas

À retenir :

L'OOP n'est pas une obligation en Python. Pour des scripts simples, des fonctions suffisent. La POO devient utile quand on modélise des entités complexes avec état et comportements, ou quand on travaille en équipe sur de grandes bases de code.

CLASSE & OBJET — LES BASES

- **Classe** : modèle / plan de construction
- **Objet (instance)** : réalisation concrète de la classe
- `__init__` : méthode constructeur, appelée à la création
- `self` : référence à l'objet courant (obligatoire en premier paramètre)

```

class Utilisateur:
    """Représente un utilisateur de l'application."""

    def __init__(self, nom, email, age):
        self.nom = nom          # attribut d'instance
        self.email = email
        self.age = age
        self.actif = True       # valeur par défaut

    def se_connecter(self):
        """Simule une connexion."""
        if self.actif:
            print(f"{self.nom} est connecté(e).")
        else:
            print(f"Compte désactivé pour {self.nom}.")

# Créer des objets (instances)
alice = Utilisateur("Alice", "alice@exemple.fr", 30)
bob = Utilisateur("Bob", "bob@exemple.fr", 25)

alice.se_connecter()          # Alice est connecté(e).
print(alice.nom)              # Alice
print(alice.age)              # 30

```

MÉTHODES & REPRÉSENTATION

- `__str__` : représentation lisible par l'humain (utilisée par `print()`)
- `__repr__` : représentation technique (utilisée par l'interpréteur)
- Les méthodes d'instance accèdent à l'objet via `self`
- On peut appeler une méthode depuis une autre méthode via `self`

```

class Produit:
    def __init__(self, nom, prix, stock):
        self.nom = nom
        self.prix = prix
        self.stock = stock

    def __str__(self):
        return f"{self.nom} – {self.prix:.2f}€"

    def __repr__(self):
        return f"Produit(nom={self.nom!r}, prix={self.prix}, stock={self.stock})"

```

```

    def est_disponible(self):
        return self.stock > 0

    def vendre(self, quantite=1):
        if quantite > self.stock:
            raise ValueError(f"Stock insuffisant (disponible : {self.stock})")
        self.stock -= quantite
        print(f"Vendu {quantite}x {self.nom}. Stock restant : {self.stock}")

p = Produit("Clavier", 79.99, 15)
print(p)          # Clavier – 79.99€
print(repr(p))    # Produit(nom='Clavier', prix=79.99, stock=15)
p.vendre(3)

```

ATTRIBUTS DE CLASSE VS D'INSTANCE

- **Attribut d'instance** : propre à chaque objet, défini dans `__init__`
- **Attribut de classe** : partagé par toutes les instances
- **Méthode de classe** (`@classmethod`) : reçoit la classe via `cls`
- **Méthode statique** (`@staticmethod`) : ni `self` ni `cls`

```

class Compteur:
    # Attribut de classe – partagé
    nb_instances = 0

    def __init__(self, nom):
        self.nom = nom          # attribut d'instance
        Compteur.nb_instances += 1

    @classmethod
    def combien(cls):
        return f"Il y a {cls.nb_instances} compteur(s)."

    @staticmethod
    def valider_nom(nom):
        """Vérifie qu'un nom est non vide."""
        return isinstance(nom, str) and len(nom) > 0

c1 = Compteur("premier")
c2 = Compteur("deuxième")

```

```
print(Compteur.combien()) # Il y a 2 compteur(s).
print(Compteur.valider_nom("ok")) # True
```

À retenir :

Un cas d'usage courant de *@classmethod* est de créer des constructeurs alternatifs :

Utilisateur.depuis_csv(ligne) ou *Produit.depuis_dict(d)* .

Héritage & polymorphisme

HÉRITAGE SIMPLE

- Une classe enfant hérite de tous les attributs et méthodes du parent
- `super()` : appelle la méthode du parent
- On peut surcharger (override) une méthode du parent
- `isinstance(obj, Classe)` : vérifie la hiérarchie

```
class Animal:
    def __init__(self, nom, espece):
        self.nom = nom
        self.espece = espece

    def parler(self):
        return "..."

    def __str__(self):
        return f"{self.nom} ({self.espece})"

class Chien(Animal):
    def __init__(self, nom, race):
        super().__init__(nom, "Canis lupus familiaris")
        self.race = race

    def parler(self): # Surcharge
        return "Woof !"

    def rapporter(self, objet):
        return f"{self.nom} rapporte {objet} !"

rex = Chien("Rex", "Berger Allemand")
print(rex) # Rex (Canis lupus familiaris)
print(rex.parler()) # Woof !
```

```
print(isinstance(rex, Chien)) # True
print(isinstance(rex, Animal)) # True
```

POLYMORPHISME & DUCK TYPING

- Polymorphisme : des objets de classes différentes répondent à la même interface
- Duck typing : "Si ça cancanne comme un canard, c'est un canard"
- Python ne requiert pas d'interface explicite — seul le comportement compte
- Les ABC (Abstract Base Classes) permettent de formaliser des contrats

```
class Cercle:
    def __init__(self, rayon):
        self.rayon = rayon

    def aire(self):
        import math
        return math.pi * self.rayon ** 2

class Rectangle:
    def __init__(self, largeur, hauteur):
        self.largeur = largeur
        self.hauteur = hauteur

    def aire(self):
        return self.largeur * self.hauteur

class Triangle:
    def __init__(self, base, hauteur):
        self.base = base
        self.hauteur = hauteur

    def aire(self):
        return 0.5 * self.base * self.hauteur

# Polymorphisme en action
formes = [Cercle(5), Rectangle(4, 6), Triangle(3, 8)]
for forme in formes:
    print(f"Aire : {forme.aire():.2f}")
```

MÉTHODES SPÉCIALES (DUNDER METHODS)

- Permettent à vos objets de se comporter comme des types natifs

- `__len__` : `len(objet)`
- `__eq__`, `__lt__`, `__gt__` : comparaisons `==`, `<`, `>`
- `__add__`, `__mul__` : opérateurs `+`, `*`
- `__contains__` : opérateur `in`
- `__iter__`, `__next__` : rendre un objet itérable

```
class Panier:
    def __init__(self):
        self._articles = []

    def ajouter(self, article):
        self._articles.append(article)

    def __len__(self):
        return len(self._articles)

    def __contains__(self, article):
        return article in self._articles

    def __iter__(self):
        return iter(self._articles)

    def __add__(self, autre):
        nouveau = Panier()
        nouveau._articles = self._articles + autre._articles
        return nouveau

    def __repr__(self):
        return f"Panier({self._articles})"

p = Panier()
p.ajouter("pomme")
p.ajouter("pain")
print(len(p))          # 2
print("pomme" in p)    # True
for article in p:
    print(article)
```

À retenir :

La liste complète des méthodes spéciales est dans la documentation officielle Python sous "Data Model". Elles permettent d'intégrer vos classes dans tous les protocoles Python (tri, hash, contexte with, etc.).

PROPRIÉTÉS AVEC @PROPERTY

- `@property` : transforme une méthode en attribut accessible en lecture
- `@nom.setter` : contrôle l'affectation avec validation
- Permet d'encapsuler la logique derrière une interface simple
- Pas besoin de parenthèses lors de l'accès

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius    # Convention : _ = "privé"

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, valeur):
        if valeur < -273.15:
            raise ValueError("Température inférieure au zéro absolu !")
        self._celsius = valeur

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @property
    def kelvin(self):
        return self._celsius + 273.15

t = Temperature(100)
print(t.celsius)    # 100
print(t.fahrenheit) # 212.0
print(t.kelvin)     # 373.15

t.celsius = -300    # ValueError !
```

Bonnes pratiques OOP

INTRODUCTION AUX PRINCIPES SOLID

- S — Single Responsibility : une classe, une responsabilité
- O — Open/Closed : ouvert à l'extension, fermé à la modification
- L — Liskov Substitution : les sous-classes remplacent les parents
- I — Interface Segregation : interfaces spécifiques plutôt que génériques
- D — Dependency Inversion : dépendre des abstractions, pas des implémentations

```
# Mauvaise pratique : classe qui fait trop de choses
class GestionnaireUtilisateur:
    def creer_utilisateur(self): ...
    def envoyer_email(self): ... # Responsabilité email !
    def sauvegarder_en_bdd(self): ... # Responsabilité DB !
    def generer_rapport_pdf(self): ... # Responsabilité PDF !

# Bonne pratique : séparation des responsabilités
class Utilisateur: ... # Données et logique métier
class ServiceEmail: ... # Envoi d'emails
class RepositoryUtilisateur: ... # Persistance
class GenerateurRapport: ... # Génération de rapports
```

DATACLASSES

- Décorateur `@dataclass` (Python 3.7+) : génère automatiquement `__init__`, `__repr__`, `__eq__`
- Idéal pour les classes qui stockent principalement des données
- Option `frozen=True` : rend l'instance immuable
- Option `order=True` : génère les opérateurs de comparaison

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Point:
    x: float
    y: float

    def distance_origine(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5
```

```

@dataclass
class Etudiant:
    nom: str
    prenom: str
    notes: List[float] = field(default_factory=list)
    actif: bool = True

    @property
    def moyenne(self):
        return sum(self.notes) / len(self.notes) if self.notes else 0.0

p = Point(3.0, 4.0)
print(p)          # Point(x=3.0, y=4.0)
print(p.distance_origine()) # 5.0

e = Etudiant("Dupont", "Alice", [14, 16, 12])
print(e.moyenne)  # 14.0

```

À retenir :

Pour des cas encore plus avancés, la bibliothèque *pydantic* offre des dataclasses avec validation des types et sérialisation JSON automatique. Très utilisée avec FastAPI.

ORGANISATION EN MODULES & PACKAGES

- Un **module** = un fichier `.py`
- Un **package** = un dossier contenant `__init__.py`
- Favorise la réutilisation, la lisibilité, les tests indépendants
- Règle : une classe par fichier (conventions PEP 8)

```

mon_projet/
├── __init__.py
├── modeles/
│   ├── __init__.py
│   ├── utilisateur.py    # class Utilisateur
│   └── produit.py        # class Produit
├── services/
│   ├── __init__.py
│   ├── auth.py           # class ServiceAuth
│   └── email.py          # class ServiceEmail
└── main.py

```

```
# modeles/utilisateur.py
class Utilisateur:
    ...

# main.py
from modeles.utilisateur import Utilisateur
from services.auth import ServiceAuth

utilisateur = Utilisateur("Alice", "alice@exemple.fr")
auth = ServiceAuth()
auth.connecter(utilisateur)
```

À retenir :

Le fichier `__init__.py` peut rester vide ou exposer une API publique du package via `from .module import Classe`. Cela permet des imports plus concis : `from mon_package import Classe` au lieu de `from mon_package.module import Classe`.

Récapitulatif du Module 3

CE QUE VOUS AVEZ APPRIS

- Classe & objet : `class`, `__init__`, `self`, attributs d'instance
- Méthodes : d'instance, de classe (`@classmethod`), statiques
- Héritage : `class Enfant(Parent)`, `super()`, `override`
- Polymorphisme : duck typing, même interface, comportements différents
- Dunder methods : `__str__`, `__repr__`, `__len__`, `__eq__` ...
- Propriétés : `@property`, `@setter` pour encapsuler la logique
- Dataclasses : génération automatique du boilerplate
- Packaging : organisation en modules et packages

Module 4 — Manipulation de données avec Python

- Comprendre l'intérêt de NumPy pour le calcul numérique performant
- Créer et manipuler des tableaux NumPy (`ndarray`)
- Charger, explorer, filtrer et transformer des données avec Pandas

- Nettoyer un jeu de données (valeurs manquantes, doublons, types)

NumPy — Calcul numérique

POURQUOI NUMPY ?

- Les listes Python sont lentes pour le calcul numérique (boucles Python, types dynamiques)
- NumPy fournit le type `ndarray` : tableau homogène, contigu en mémoire
- Opérations vectorisées : calcul en C sans boucle Python → 10 à 100× plus rapide
- Base de tout l'écosystème scientifique Python (Pandas, scikit-learn, TensorFlow...)

```
pip install numpy
```

```
import numpy as np

# Liste Python (lente)
import time
liste = list(range(1_000_000))
debut = time.time()
result = [x * 2 for x in liste]
print(f"Liste : {time.time() - debut:.4f}s")

# NumPy (rapide)
arr = np.arange(1_000_000)
debut = time.time()
result = arr * 2
print(f"NumPy : {time.time() - debut:.4f}s")
```

CRÉER DES TABLEAUX NUMPY

```
import numpy as np

# Depuis une liste Python
a = np.array([1, 2, 3, 4, 5])
m = np.array([[1, 2, 3], [4, 5, 6]]) # 2D

# Tableaux prédéfinis
zeros = np.zeros((3, 4)) # 3 lignes, 4 colonnes de 0.0
uns = np.ones((2, 3)) # de 1.0
identite = np.eye(3) # Matrice identité 3x3
```

```

# Séquences
suite = np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]
lineaire = np.linspace(0, 1, 5) # [0.0, 0.25, 0.5, 0.75, 1.0]

# Aléatoire
aleatoire = np.random.rand(3, 3) # Uniformément [0, 1)
normal = np.random.randn(1000)   # Distribution normale

# Informations sur le tableau
print(m.shape)    # (2, 3)
print(m.dtype)    # int64
print(m.ndim)     # 2
print(m.size)     # 6

```

INDEXING, SLICING & BROADCASTING

```

import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Accès par index
print(a[0, 0])    # 1
print(a[1, 2])    # 6
print(a[-1, -1])  # 9

# Slicing
print(a[0, :])    # [1 2 3] – première ligne
print(a[:, 1])    # [2 5 8] – deuxième colonne
print(a[:2, 1:])  # [[2 3] [5 6]]

# Indexation conditionnelle
print(a[a > 5])    # [6 7 8 9]
a[a < 3] = 0       # Modifier les éléments < 3

# Broadcasting – opérations sur des formes différentes
b = np.array([1, 2, 3])
print(a + b)       # b est "étiré" pour correspondre à chaque ligne de a

```

À retenir :

Le broadcasting est l'une des fonctionnalités les plus puissantes de NumPy. Il permet d'appliquer des opérations entre tableaux de formes différentes sans boucle ni copie mémoire.

FONCTIONS MATHÉMATIQUES & STATISTIQUES

```
import numpy as np

data = np.array([12, 7, 18, 4, 21, 9, 15, 6, 13, 11])

# Statistiques descriptives
print(f"Moyenne : {np.mean(data):.2f}")
print(f"Médiane : {np.median(data):.2f}")
print(f"Écart-type: {np.std(data):.2f}")
print(f"Min / Max : {np.min(data)} / {np.max(data)}")
print(f"Somme : {np.sum(data)}")

# Index des extrêmes
print(f"Index min : {np.argmin(data)}")
print(f"Index max : {np.argmax(data)}")

# Tri
print(np.sort(data))
print(np.argsort(data)) # Indices qui trieraient le tableau

# Opérations mathématiques
print(np.sqrt(data))
print(np.log(data))
print(np.exp(data[:3]))
```

Pandas — Analyse de données

INTRODUCTION À PANDAS

- **Series** : tableau 1D avec index (colonne)
- **DataFrame** : tableau 2D avec index et colonnes nommées (la structure centrale)
- Pandas = NumPy + index + gestion des données manquantes + I/O

```
import pandas as pd
import numpy as np
```

```

# Series
notes = pd.Series([14, 17, 11, 19, 8],
                   index=["Alice", "Bob", "Charlie", "Diana", "Eve"])
print(notes["Alice"])      # 14
print(notes[notes > 12])   # Filtrer

# DataFrame depuis un dictionnaire
data = {
    "nom": ["Alice", "Bob", "Charlie", "Diana"],
    "age": [30, 25, 35, 28],
    "ville": ["Paris", "Lyon", "Paris", "Marseille"],
    "salaire": [45000, 38000, 52000, 41000]
}
df = pd.DataFrame(data)
print(df)
print(df.dtypes)

```

À retenir :

L'index d'un DataFrame peut être numérique (par défaut 0, 1, 2...), des chaînes de caractères, des dates, etc. L'index joue un rôle clé pour l'alignement des données lors des opérations.

CHARGEMENT DE DONNÉES

```

import pandas as pd

# CSV (le plus courant)
df = pd.read_csv("ventes.csv",
                 sep=";",          # séparateur
                 encoding="utf-8",
                 parse_dates=["date"], # colonnes à parser en date
                 dtype={"code": str}) # forcer un type

# Excel
df_excel = pd.read_excel("rapport.xlsx",
                        sheet_name="Feuille1",
                        skiprows=2)      # ignorer les 2 premières lignes

# JSON
df_json = pd.read_json("donnees.json")

# Depuis une URL
df_web = pd.read_csv("https://exemple.com/data.csv")

```

```
# Sauvegarder
df.to_csv("resultat.csv", index=False, sep=";")
df.to_excel("resultat.xlsx", sheet_name="Données", index=False)
```

EXPLORATION D'UN DATAFRAME

```
import pandas as pd

df = pd.read_csv("ventes.csv")

# Aperçu général
print(df.head(5))      # 5 premières lignes
print(df.tail(3))      # 3 dernières lignes
print(df.sample(5))    # 5 lignes aléatoires

# Informations structurelles
print(df.shape)        # (nb_lignes, nb_colonnes)
print(df.dtypes)       # Type de chaque colonne
print(df.info())       # Résumé complet (types + valeurs non-nulles)

# Statistiques descriptives
print(df.describe())   # count, mean, std, min, quartiles, max

# Valeurs uniques
print(df["ville"].unique()) # Valeurs distinctes
print(df["ville"].nunique()) # Nombre de valeurs distinctes
print(df["ville"].value_counts()) # Fréquence de chaque valeur

# Colonnes et index
print(df.columns.tolist())
print(df.index)
```

SÉLECTION & FILTRAGE

```
import pandas as pd

df = pd.read_csv("employes.csv")

# Sélection de colonnes
noms = df["nom"]          # Series
sous_ensemble = df[["nom", "salaire"]] # DataFrame
```

```

# .loc – par étiquette
ligne = df.loc[0]          # Première ligne
cellule = df.loc[0, "nom"]  # Valeur précise
bloc = df.loc[0:5, "nom":"age"]  # Bloc (inclus/inclus)

# .iloc – par position numérique
ligne = df.iloc[0]         # Première ligne
cellule = df.iloc[0, 1]    # Ligne 0, colonne 1
bloc = df.iloc[:5, 1:3]    # Lignes 0-4, colonnes 1-2

# Filtrage conditionnel
seniors = df[df["age"] > 40]
parisiens = df[df["ville"] == "Paris"]
haut_salaire = df[(df["salaire"] > 50000) & (df["ville"] == "Paris")]
selection = df[df["ville"].isin(["Paris", "Lyon"])]

```

TRANSFORMATION DES DONNÉES

```

import pandas as pd

df = pd.read_csv("ventes.csv")

# Appliquer une fonction sur une colonne
df["salaire_net"] = df["salaire"] * 0.77
df["prenom_maj"] = df["prenom"].str.upper()

# apply – fonction personnalisée
def categoriser_age(age):
    if age < 30: return "Junior"
    elif age < 45: return "Senior"
    else: return "Expert"

df["categorie"] = df["age"].apply(categoriser_age)

# groupby – agréger par groupe
par_ville = df.groupby("ville")["salaire"].mean()
stats = df.groupby("ville").agg(
    nb_employes=("nom", "count"),
    salaire_moyen=("salaire", "mean"),
    age_moyen=("age", "mean")
)

```

```
# pivot_table
pivot = df.pivot_table(
    values="ventes",
    index="region",
    columns="trimestre",
    aggfunc="sum"
)
```

À retenir :

groupby est l'équivalent Python du *GROUP BY* SQL. La méthode *.agg()* permet d'appliquer plusieurs fonctions d'agrégation en une seule opération, ce qui est très efficace.

NETTOYAGE DE DONNÉES

```
import pandas as pd
import numpy as np

df = pd.read_csv("donnees_brutes.csv")

# Valeurs manquantes
print(df.isnull().sum())      # Nb de NaN par colonne
print(df.isnull().sum() / len(df) * 100) # En %

df.dropna()                  # Supprimer les lignes avec NaN
df.dropna(subset=["email"])   # Seulement si 'email' est NaN
df.fillna(0)                 # Remplacer NaN par 0
df["age"].fillna(df["age"].mean(), inplace=True) # Par la moyenne

# Doublons
print(df.duplicated().sum())  # Nb de doublons
df.drop_duplicates(inplace=True)
df.drop_duplicates(subset=["email"], keep="first")

# Conversion de types
df["date"] = pd.to_datetime(df["date"])
df["age"] = pd.to_numeric(df["age"], errors="coerce") # NaN si invalide
df["code"] = df["code"].astype(str)
```

Récapitulatif du Module 4

CE QUE VOUS AVEZ APPRIS

- NumPy : ndarray, opérations vectorisées, indexing, broadcasting, statistiques
- Pandas Series : tableau 1D indexé
- Pandas DataFrame : tableau 2D, la structure centrale
- Chargement : `read_csv`, `read_excel`, `read_json`
- Exploration : `head`, `info`, `describe`, `value_counts`
- Sélection : `loc` (étiquettes), `iloc` (positions), filtres booléens
- Transformation : `apply`, `groupby`, `agg`, `pivot_table`
- Nettoyage : `isnull`, `fillna`, `dropna`, `drop_duplicates`, `astype`

Module 5 — Bibliothèques pour scripts & automatisation

- Écrire des scripts Python robustes et bien structurés
- Gérer les arguments en ligne de commande avec `argparse`
- Implémenter du logging professionnel et la gestion de configuration
- Automatiser des tâches : fichiers, web, expressions régulières, subprocess
- Mettre en place des tests et packager son projet

Scripts robustes

STRUCTURE D'UN BON SCRIPT PYTHON

- `if __name__ == "__main__"` : code exécuté seulement en direct (pas à l'import)
- Séparer la logique métier des entrées/sorties
- Toujours documenter avec des docstrings
- Utiliser des fonctions courtes avec une seule responsabilité

```
#!/usr/bin/env python3
```

```
"""
```

```
traitement_csv.py – Script de traitement de fichiers CSV.
```

Usage:

```
python traitement_csv.py --input data.csv --output result.csv
```

```
"""
import argparse
import logging
from pathlib import Path

def configurer_logging(verbose: bool = False):
    niveau = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=niveau,
        format="%(asctime)s [%(levelname)s] %(message)s"
    )

def traiter_fichier(input_path: Path, output_path: Path) -> int:
    """Traite un fichier CSV et retourne le nb de lignes traitées."""
    logging.info(f"Traitement de {input_path}")
    # ... logique métier ...
    return 0

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", required=True, type=Path)
    parser.add_argument("--output", required=True, type=Path)
    parser.add_argument("--verbose", "-v", action="store_true")
    args = parser.parse_args()
    configurer_logging(args.verbose)
    traiter_fichier(args.input, args.output)

if __name__ == "__main__":
    main()
```

ARGUMENTS EN LIGNE DE COMMANDE AVEC ARGPARSE

- Module standard Python (pas d'installation)
- Génère automatiquement l'aide (`--help`)
- Types, valeurs par défaut, arguments obligatoires ou optionnels
- Sous-commandes avec `add_subparsers()`

```
import argparse
```

```

parser = argparse.ArgumentParser(
    description="Outil d'analyse de données",
    epilog="Exemple: python outil.py data.csv --format excel --limit 100"
)

# Arguments positionnels (obligatoires)
parser.add_argument("fichier", help="Fichier CSV à traiter")

# Arguments optionnels
parser.add_argument("--format", choices=["csv", "excel", "json"],
                    default="csv", help="Format de sortie")
parser.add_argument("--limit", type=int, default=None,
                    help="Nombre max de lignes")
parser.add_argument("--verbose", "-v", action="store_true",
                    help="Mode verbeux")
parser.add_argument("--output", "-o", type=str, default="output.csv",
                    help="Fichier de sortie")

args = parser.parse_args()
print(args.fichier) # "data.csv"
print(args.format) # "csv"
print(args.verbose) # True/False

```

À retenir :

argparse gère automatiquement `--help` et les erreurs de syntaxe. Pour des cas plus avancés, la bibliothèque *click* (tierce) offre une API encore plus élégante basée sur des décorateurs.

LOGGING PROFESSIONNEL

- Module `logging` : standard Python, très complet
- Niveaux : `DEBUG` < `INFO` < `WARNING` < `ERROR` < `CRITICAL`
- Handler : destination des logs (console, fichier, réseau...)
- Formatter : format des messages
- Logger par module : bonne pratique pour les projets multi-fichiers

```

import logging
from pathlib import Path

# Configuration de base
logging.basicConfig(
    level=logging.DEBUG,

```

```

    format="%asctime)s [%(levelname)-8s] %(name)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    handlers=[
        logging.StreamHandler(), # Console
        logging.FileHandler("app.log", encoding="utf-8") # Fichier
    ]
)

logger = logging.getLogger(__name__) # Logger du module courant

logger.debug("Détail de debug")
logger.info("Traitement démarré")
logger.warning("Fichier non trouvé, utilisation du cache")
logger.error("Connexion échouée")

try:
    1 / 0
except ZeroDivisionError:
    logger.exception("Erreur inattendue") # Log + stacktrace

```

GESTION DE CONFIGURATION

- Séparation du code et de la configuration : principe fondamental
- `.env` + `python-dotenv` : variables d'environnement (secrets, URLs)
- `configparser` : fichiers `.ini` (configuration applicative)
- Ne jamais committer de secrets dans Git

```

# Fichier .env (jamais dans Git !)
# DATABASE_URL=postgresql://user:pass@localhost/mydb
# API_KEY=sk-xxx
# DEBUG=true

from dotenv import load_dotenv
import os

load_dotenv() # Charge les variables du fichier .env

db_url = os.getenv("DATABASE_URL")
api_key = os.getenv("API_KEY")
debug = os.getenv("DEBUG", "false").lower() == "true"

# Avec configparser (pour config non-sensible)

```

```
import configparser

config = configparser.ConfigParser()
config.read("config.ini")

host = config["database"]["host"]
port = config.getint("database", "port")
```

À retenir :

La bibliothèque *pydantic-settings* permet de valider et typer les variables d'environnement automatiquement. Très utilisée dans les projets FastAPI pour la configuration applicative.

Automatisation & interaction système

MANIPULATION AVANCÉE DE FICHIERS

```
import shutil
import os
from pathlib import Path

source = Path("rapport_source.pdf")
dest = Path("archives/rapport_2024.pdf")

# Copie et déplacement
shutil.copy(source, dest)          # Copie fichier
shutil.copytree("src/", "backup/") # Copie arborescence
shutil.move("tmp.txt", "final.txt") # Déplacement/renommage
shutil.rmtree("dossier_temp/")     # Supprime dossier entier

# Recherche avec glob
for f in Path(".").glob("**/*.csv"): # Récursif
    print(f)

for f in Path("logs").glob("*.log"):
    if f.stat().st_size > 10 * 1024 * 1024: # > 10 Mo
        f.unlink()

# Parcours d'arborescence
for racine, dossiers, fichiers in os.walk("projet/"):
    for fichier in fichiers:
```

```
chemin = Path(racine) / fichier
print(chemin)
```

EXPRESSIONS RÉGULIÈRES

- Module `re` : moteurs d'expressions régulières
- `re.search()` : cherche dans la chaîne (retourne le premier match)
- `re.match()` : cherche uniquement en début de chaîne
- `re.findall()` : retourne toutes les correspondances
- `re.sub()` : remplacement

```
import re

texte = "Contactez alice@exemple.fr ou bob.martin@corp.com pour info."

# Trouver des emails
pattern_email = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
emails = re.findall(pattern_email, texte)
# ['alice@exemple.fr', 'bob.martin@corp.com']

# Valider un numéro de téléphone
tel = "06 12 34 56 78"
match = re.match(r"^0[1-9](\s?\d{2}){4}$", tel)
if match:
    print("Numéro valide")

# Extraire des groupes
log = "2024-03-15 14:23:45 ERROR Connexion échouée"
m = re.search(r"(\d{4}-\d{2}-\d{2}) (\d{2}:\d{2}:\d{2}) (\w+) (.+)", log)
if m:
    date, heure, niveau, message = m.groups()

# Remplacement
propre = re.sub(r"\s+", " ", "texte avec espaces")
```

REQUÊTES HTTP AVEC REQUESTS

```
import requests

# GET simple
response = requests.get("https://api.example.com/users")
print(response.status_code) # 200
```

```

print(response.headers["Content-Type"])
data = response.json() # Parse le JSON

# GET avec paramètres
params = {"page": 1, "limit": 20, "ville": "Paris"}
r = requests.get("https://api.example.com/users", params=params)

# POST avec JSON
payload = {"nom": "Alice", "email": "alice@exemple.fr"}
headers = {"Authorization": "Bearer mon_token_api"}
r = requests.post(
    "https://api.example.com/users",
    json=payload,
    headers=headers,
    timeout=10 # Toujours définir un timeout !
)

# Gestion des erreurs
r.raise_for_status() # Lève une exception si code HTTP >= 400

if r.status_code == 201:
    utilisateur = r.json()

```

À retenir :

requests n'est pas dans la bibliothèque standard. Installer avec `pip install requests`. Pour les projets asynchrones ou les performances, *httpx* et *aiohttp* sont des alternatives modernes.

INTERACTION SYSTÈME AVEC SUBPROCESS

```

import subprocess
from pathlib import Path

# Exécuter une commande simple
result = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(result.stdout) # Python 3.12.x
print(result.returncode) # 0 = succès

# Commande avec shell (attention : risque d'injection)

```

```

result = subprocess.run(
    "ls -la *.py",
    shell=True, capture_output=True, text=True
)

# Vérifier le succès
try:
    subprocess.run(
        ["git", "pull"],
        check=True,      # Lève CalledProcessError si code != 0
        capture_output=True,
        text=True
    )
except subprocess.CalledProcessError as e:
    print(f"Erreur git : {e.stderr}")

```

Qualité & packaging

TESTS UNITAIRES AVEC PYTEST

- `unittest` : module standard, orienté classes
- `pytest` : outil de test de facto, syntaxe plus simple
- Nommage : fichiers `test_*.py`, fonctions `test_*`
- `assert` : assertions de test
- Fixtures : données et contextes partagés entre tests

```

# fichier: test_calculs.py
import pytest

from mon_projet.calculs import calculer_imc, valider_age

def test_imc_normal():
    imc = calculer_imc(70, 1.75)
    assert abs(imc - 22.86) < 0.01

def test_imc_taille_nulle():
    with pytest.raises(ValueError):
        calculer_imc(70, 0)

def test_age_valide():
    assert valider_age(25) == 25

```

```

def test_age_negatif():
    with pytest.raises(ValueError):
        valider_age(-5)

@pytest.mark.parametrize("poids, taille, attendu", [
    (70, 1.75, 22.86),
    (90, 1.80, 27.78),
    (50, 1.60, 19.53),
])

def test_imc_parametrique(poids, taille, attendu):
    assert abs(calculer_imc(poids, taille) - attendu) < 0.01

```

```

pytest                # Lance tous les tests
pytest -v             # Mode verbeux
pytest test_calculs.py -k "imc" # Filtrer par nom

```

À retenir :

Un bon test vérifie un seul comportement, est indépendant des autres tests, et peut s'exécuter dans n'importe quel ordre. Viser un taux de couverture (coverage) de 70-80% pour un projet sérieux.

GESTION DES DÉPENDANCES

```
# Créer et activer un environnement virtuel

python -m venv .venv
source .venv/bin/activate          # Linux/Mac
.venv\Scripts\activate            # Windows

# Installer des dépendances
pip install requests pandas numpy

# Exporter les dépendances
pip freeze > requirements.txt

# Installer depuis requirements.txt
pip install -r requirements.txt

# Vérifier les dépendances obsolètes
pip list --outdated

# Mettre à jour une dépendance
pip install --upgrade requests
```

PACKAGING BASIQUE D'UN PROJET PYTHON

```
mon_projet/
├─ src/
│   └─ mon_projet/
│       ├── __init__.py
│       ├── core.py
│       └─ utils.py
├─ tests/
│   ├── test_core.py
│   └─ test_utils.py
├─ .env.example
├─ .gitignore
├─ pyproject.toml
└─ README.md
```

```
# pyproject.toml

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "mon-projet"
version = "0.1.0"
description = "Description du projet"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.31",
    "pandas>=2.0",
]

[project.scripts]
mon-outil = "mon_projet.core:main" # Point d'entrée CLI
```

À retenir :

Un projet bien structuré avec `pyproject.toml` peut être publié sur PyPI avec

`pip install build && python -m build && twine upload dist/*`. Cela rend votre outil installable par n'importe qui avec `pip install mon-projet`.

Récapitulatif du Module 5

CE QUE VOUS AVEZ APPRIS

- Structure de script : `if __name__ == "__main__"`, organisation
- `argparse` : arguments CLI typés, aide automatique
- `logging` : niveaux, handlers, formatters, logger par module
- Configuration : `.env`, `python-dotenv`, `configparser`
- Fichiers avancés : `shutil`, `glob`, `os.walk`
- Expressions régulières : `re.search`, `findall`, `sub`, groupes
- HTTP : `requests`, GET/POST, JSON, gestion des erreurs
- `subprocess` : exécuter des commandes système
- Tests : `pytest`, `assert`, `parametrize`, `fixtures`
- Packaging : `requirements.txt`, `pyproject.toml`, structure de projet

Module 6 — Ouvertures & perspectives

Objectifs pédagogiques

À l'issue de ce module, les participants seront capables de :

- Identifier les principaux écosystèmes Python (data science, web, cloud)
- Comprendre le workflow d'un projet de machine learning avec scikit-learn
- Créer une API REST minimale avec Flask et consommer des APIs tierces
- Manipuler une base de données SQLite en Python natif et via un ORM
- Appliquer les bonnes pratiques de développement professionnel (venv, git, CI/CD)
- Identifier les ressources clés pour poursuivre leur apprentissage en autonomie

Python pour la data science

L'ÉCOSYSTÈME DATA SCIENCE PYTHON

- Python est le langage dominant en data science depuis ~2015
- Un ensemble de bibliothèques couvre tout le pipeline analytique
- NumPy — tableaux multidimensionnels, calcul numérique haute performance
- pandas — manipulation de données tabulaires (DataFrames)
- Matplotlib / Seaborn — visualisation de données statique
- scikit-learn — machine learning classique (classification, régression, clustering)
- Jupyter — environnement interactif de développement et d'exploration

Ressource de référence : <https://scipy-lectures.org> — une introduction complète à l'écosystème scientifique Python. Pour une vue d'ensemble en français : <https://www.fun-mooc.fr> (chercher “Python data science”).

JUPYTER NOTEBOOK — INTERFACE ET CELLULES

- Jupyter Notebook = interface web pour écrire du code Python interactivement
- Le document est composé de **cellules** indépendantes
- Types de cellules : **Code** (Python), **Markdown** (texte formaté), **Raw**
- Exécution : **Shift+Enter** exécute et passe à la suivante
- **Ctrl+Enter** exécute sans avancer
- Les résultats (graphiques, tableaux) s'affichent directement sous la cellule
- Jupyter Lab = version plus moderne avec onglets et explorateur de fichiers

Pour installer Jupyter : `pip install jupyterlab`. Lancer avec `jupyter lab` dans votre terminal. JupyterLab est accessible sur `http://localhost:8888` par défaut. Les fichiers notebooks ont l'extension `.ipynb` (JSON sous le capot).

JUPYTER — MAGIC COMMANDS ET BONNES PRATIQUES

```
# Magic commands utiles
%timeit sum(range(1000))      # mesure le temps d'exécution
%matplotlib inline           # affiche les graphiques dans le notebook
%run mon_script.py           # exécute un fichier Python externe
%who                          # liste les variables en mémoire

# Aliases conventionnels — toujours en haut du notebook
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Bonnes pratiques
# - Structurez avec des cellules Markdown pour documenter
# - Nommez les notebooks : 01-exploration.ipynb, 02-analyse.ipynb
# - Redémarrez le kernel régulièrement pour vérifier la reproductibilité
```

- L'ordre d'exécution des cellules compte — l'état du kernel est global
- "Restart & Run All" valide la reproductibilité complète du notebook
- Évitez les effets de bord entre cellules

VISUALISATION DE DONNÉES AVEC MATPLOTLIB

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
```

```

y = np.sin(x)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Graphique linéaire
axes[0].plot(x, y, color='steelblue', label='sin(x)')
axes[0].set_title('Courbe sinusoïde')
axes[0].set_xlabel('x')
axes[0].legend()

# Histogramme
data = np.random.normal(0, 1, 1000)
axes[1].hist(data, bins=30, color='salmon', edgecolor='white')
axes[1].set_title('Distribution normale')

plt.tight_layout()
plt.show()

```

Graphiques courants : `plot()` (ligne), `bar()` (barres), `scatter()` (nuage de points), `hist()` (histogramme), `boxplot()` (boîte à moustaches). La galerie officielle recense des centaines d'exemples avec le code source : <https://matplotlib.org/stable/gallery/>

VISUALISATION AVEC SEABORN

```

import seaborn as sns
import matplotlib.pyplot as plt

# Seaborn fournit des jeux de données d'exemple
tips = sns.load_dataset('tips')

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

```

```
# Distribution avec estimation de densité (KDE)
sns.histplot(tips['total_bill'], kde=True, ax=axes[0])
axes[0].set_title('Distribution des additions')

# Relation entre deux variables colorée par catégorie
sns.scatterplot(
    data=tips, x='total_bill', y='tip',
    hue='sex', style='smoker', ax=axes[1]
)
axes[1].set_title('Pourboire vs Addition')

plt.tight_layout()
plt.show()
```

- Seaborn s'appuie sur Matplotlib et produit de beaux graphiques statistiques par défaut
- S'intègre nativement avec les DataFrames pandas

Seaborn excelle pour les graphiques statistiques :
`violinplot`, **`boxplot`**, **`heatmap`** (matrices de corrélation),
`pairplot` (toutes les paires de variables d'un coup). Le
paramètre **`hue`** permet de colorer selon une variable
catégorielle sans code supplémentaire.

INTRODUCTION AU MACHINE LEARNING — WORKFLOW

- Le Machine Learning = apprendre des règles à partir des données (pas de règles codées manuellement)
- Workflow standard en 5 étapes : ** 1. Données — collecter, nettoyer, explorer (EDA) ** 2. Préparation — encoder, normaliser, diviser train/test ** 3. Modèle — choisir et instancier un algorithme ** 4. Entraînement — `model.fit(X_train, y_train)` ** 5. Évaluation — `model.score(X_test, y_test)` , métriques
- Deux grandes familles : **supervisé** (données labellisées) vs **non supervisé**
- scikit-learn fournit une API uniforme pour tous ses algorithmes

**Types d'algorithmes supervisés courants :
régression linéaire (valeurs continues), régression
logistique (classification binaire), arbres de
décision, forêts aléatoires (Random Forest), SVM.
Non supervisé : K-Means (clustering), PCA
(réduction de dimension).**

MACHINE LEARNING AVEC SCIKIT-LEARN

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Données
iris = load_iris()
X, y = iris.data, iris.target

# 2. Préparation : séparation train / test (80% / 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3 & 4. Modèle + Entraînement
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 5. Évaluation
y_pred = model.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2%}")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

L'accuracy seule peut être trompeuse sur des jeux de données déséquilibrés (ex : 95% classe A, 5% classe B — un modèle qui prédit toujours A atteint 95% d'accuracy sans rien apprendre). Apprenez à interpréter la matrice de confusion :

```
from sklearn.metrics  
import ConfusionMatrixDisplay.
```

Python pour le web & les APIs

FRAMEWORKS WEB PYTHON — VUE D'ENSEMBLE

- Python propose plusieurs frameworks web matures aux positionnements distincts
- Flask — micro-framework minimaliste, très flexible, idéal pour commencer ou pour les microservices
- Django — framework “batteries included” : ORM intégré, admin auto-générée, authentification
- FastAPI — moderne, hautes performances, validation automatique, documentation Swagger intégrée
- Critères de choix : ** Taille du projet et contraintes de l'équipe ** Besoin de flexibilité vs conventions fortes ** Performance et gestion de l'asynchrone (FastAPI est async-first)

Quelques chiffres (2024) : Django et Flask dominent toujours en production. FastAPI est le framework à la croissance la plus rapide, notamment dans l'univers data/ML pour exposer des modèles (model serving). Starlette et Uvicorn sont les briques sous-jacentes de FastAPI.

FLASK — PREMIÈRE APPLICATION WEB

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Données en mémoire (simuler une base de données)
utilisateurs = [
    {"id": 1, "nom": "Alice", "role": "admin"},
    {"id": 2, "nom": "Bob", "role": "user"},
]

@app.route("/")
def accueil():
    return "<h1>Bienvenue sur mon API</h1>"

@app.route("/utilisateurs", methods=["GET"])
def lister_utilisateurs():
    return jsonify(utilisateurs)

@app.route("/utilisateurs/<int:user_id>", methods=["GET"])
def get_utilisateur(user_id):
    user = next((u for u in utilisateurs if u["id"] == user_id), None)
    if user is None:
        return jsonify({"erreur": "Introuvable"}), 404
    return jsonify(user)
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

FLASK — CRÉER ET MODIFIER DES RESSOURCES

```
from flask import Flask, jsonify, request  
  
app = Flask(__name__)  
utilisateurs = []  
compteur_id = 1  
  
@app.route("/utilisateurs", methods=["POST"])  
def creer_utilisateur():  
    global compteur_id  
    data = request.get_json()  
    if not data or "nom" not in data:  
        return jsonify({"erreur": "Champ 'nom' requis"}), 400  
    nouvel_utilisateur = {  
        "id": compteur_id,  
        "nom": data["nom"],  
        "role": data.get("role", "user")  
    }  
    utilisateurs.append(nouvel_utilisateur)  
    compteur_id += 1  
    return jsonify(nouvel_utilisateur), 201 # 201 Created  
  
@app.route("/utilisateurs/<int:user_id>", methods=["DELETE"])  
def supprimer_utilisateur(user_id):  
    global utilisateurs  
    avant = len(utilisateurs)  
    utilisateurs = [u for u in utilisateurs if u["id"] != user_id]  
    if len(utilisateurs) == avant:  
        return jsonify({"erreur": "Introuvable"}), 404  
    return "", 204 # 204 No Content
```

Pour tester des APIs localement :
(interface graphique) ou **(`pip install httpie`)** plus
lisible que curl. En production, ajoutez une vraie
base de données (SQLAlchemy), une
authentification (JWT avec `flask-jwt-extended`) et un
gestionnaire d'erreurs global avec `@app.errorhandler(404)`.

CONSOMMER UNE API TIERCE — REQUÊTES HTTP

```
import requests

BASE_URL = "https://api.github.com"
HEADERS = {
    "Accept": "application/vnd.github.v3+json",
    # "Authorization": "Bearer VOTRE_TOKEN" # endpoints privés
}

# Requête GET simple
resp = requests.get(f"{BASE_URL}/users/python", headers=HEADERS)
resp.raise_for_status() # lève une exception si erreur HTTP
user = resp.json()
print(f"Nom : {user['name']}, Repos publics : {user['public_repos']}")

# Requête avec paramètres de pagination
params = {"per_page": 5, "page": 1, "sort": "stars"}
repos = requests.get(
    f"{BASE_URL}/users/python/repos",
    headers=HEADERS,
    params=params
).json()
```

```
for repo in repos:
    print(f" - {repo['name']} ({repo['stargazers_count']} étoiles)")
```

- `raise_for_status()` transforme silencieusement un 404/500 en exception Python
- `resp.json()` parse le corps JSON en dictionnaire Python
- `params=` construit automatiquement la query string de l'URL

Authentification courante pour les APIs tierces : Bearer Token (*Authorization: Bearer <token>*), API Key (header ou query param), OAuth 2.0 (flux plus complexe). Stockez toujours vos tokens dans des variables d'environnement, jamais dans le code source.

```
import os
TOKEN = os.environ.get("GITHUB_TOKEN")
```

====

CONSOMMER UNE API — PAGINATION ET GESTION D'ERREURS

```
import requests, time

def fetch_all_pages(url, headers, params=None, max_pages=10):
    """Récupère toutes les pages d'une API paginée."""
    results = []
    params = (params or {}) | {"per_page": 100}

    for page in range(1, max_pages + 1):
        params["page"] = page
        resp = requests.get(url, headers=headers,
                             params=params, timeout=10)

        if resp.status_code == 429:  # Rate limit atteint
            wait = int(resp.headers.get("Retry-After", 60))
            print(f"Rate limit – attente {wait}s")
            time.sleep(wait)
            continue

    resp.raise_for_status()
```

```

    page_data = resp.json()
    if not page_data:
        # Plus aucune donnée
        break
    results.extend(page_data)

return results

```

- Toujours gérer le **rate limiting** (code HTTP 429)
- Ajouter un `timeout` pour éviter les blocages indéfinis
- Préférer `extend()` à des `append()` imbriqués pour agréger les pages

Bibliothèques utiles : `httplib2` (alternative moderne à `requests`, supporte **async**), `tenacity` (décorateur de **retry** avec **backoff** exponentiel). Pour les APIs REST complexes, des SDKs clients générés automatiquement depuis une spec OpenAPI existent (`openapi-generator`).

Autres horizons Python

PYTHON & LES BASES DE DONNÉES — SQLITE3 NATIF

```

import sqlite3
from contextlib import contextmanager

@contextmanager
def get_db(path="app.db"):
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row # accès par nom de colonne
    try:
        yield conn
        conn.commit()
    except Exception:

```

```

        conn.rollback()
    raise
    finally:
        conn.close()

# Création de table
with get_db() as conn:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS produits (
            id    INTEGER PRIMARY KEY AUTOINCREMENT,
            nom   TEXT    NOT NULL,
            prix  REAL    NOT NULL,
            stock INTEGER DEFAULT 0
        )
    """)

# Insertion sécurisée avec placeholders
conn.execute(
    "INSERT INTO produits (nom, prix, stock) VALUES (?, ?, ?)",
    ("Clavier mécanique", 89.99, 50)
)

```

sqlite3 est approprié pour des applications mono-utilisateur ou à faible charge (outils CLI, scripts d'analyse, prototypes). Pour des applications multi-utilisateurs concurrentes, préférez PostgreSQL ou MySQL avec un pool de connexions. Le fichier `.db` est un fichier binaire portable : vous pouvez l'ouvrir avec l'outil “DB Browser for SQLite”.

PYTHON & LES BASES DE DONNÉES — SQLALCHEMY ORM

```

from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import DeclarativeBase, Session

```

```

engine = create_engine("sqlite:///app.db", echo=False)

class Base(DeclarativeBase):
    pass

class Produit(Base):
    __tablename__ = "produits"
    id = Column(Integer, primary_key=True, autoincrement=True)
    nom = Column(String(100), nullable=False)
    prix = Column(Float, nullable=False)
    stock = Column(Integer, default=0)

    def __repr__(self):
        return f"<Produit(nom={self.nom!r}, prix={self.prix})>"

Base.metadata.create_all(engine) # crée les tables si absentes

# CRUD via l'ORM
with Session(engine) as session:
    clavier = Produit(nom="Clavier mécanique", prix=89.99, stock=50)
    session.add(clavier)
    session.commit()

    resultats = session.query(Produit).filter(Produit.prix < 100).all()
    for p in resultats:
        print(p)

```

SQLAlchemy 2.x (style moderne) utilise `select()` au lieu de `session.query()`. Alembic (compagnon officiel de SQLAlchemy) gère les migrations de schéma de façon incrémentale : `alembic revision --autogenerate -m "ajouter colonne description"` puis `alembic upgrade head`.

PYTHON & LE CLOUD — APERÇU DES SDKS

```
# --- AWS avec boto3 ---
import boto3

s3 = boto3.client("s3", region_name="eu-west-1")

buckets = s3.list_buckets()["Buckets"]
for b in buckets:
    print(b["Name"])

s3.upload_file("rapport.pdf", "mon-bucket", "rapports/2024/rapport.pdf")

# Lire un objet S3
obj = s3.get_object(Bucket="mon-bucket", Key="config.json")
contenu = obj["Body"].read().decode("utf-8")

# --- Azure Blob Storage ---
# from azure.storage.blob import BlobServiceClient
# client = BlobServiceClient.from_connection_string(os.environ["CONN_STR"])

# --- Google Cloud Storage ---
# from google.cloud import storage
# gcs = storage.Client()
# bucket = gcs.bucket("mon-bucket")
# bucket.blob("fichier.txt").upload_from_filename("fichier.txt")
```

Authentification recommandée par cloud : AWS — IAM Roles (instances EC2/Lambda) ou `~/.aws/credentials`. Azure — Managed Identity ou `DefaultAzureCredential`. GCP — Service Account JSON ou Application Default Credentials. Installation : `pip install boto3` / `pip install azure-storage-blob` / `pip install google-cloud-storage`.

BONNES PRATIQUES — ENVIRONNEMENTS VIRTUELS

```
# Créer un environnement virtuel
python -m venv .venv

# Activer (Linux / macOS)
source .venv/bin/activate

# Activer (Windows PowerShell)
.venv\Scripts\Activate.ps1

# Installer les dépendances du projet
pip install -r requirements.txt

# Geler les dépendances actuelles
pip freeze > requirements.txt

# Désactiver l'environnement
deactivate
```

- Un environnement virtuel par projet — isolation totale des dépendances
- Ne jamais committer le dossier `.venv` (ajouter dans `.gitignore`)
- Alternative moderne : `uv` — gestionnaire de paquets 10 à 100x plus rapide que pip

Outils alternatifs de gestion d'environnement et de dépendances : (gestion des dépendances + packaging en un outil), (écosystème scientifique, gère aussi des dépendances non-Python comme des libs C), (moins populaire aujourd'hui). Le fichier `pyproject.toml` tend à remplacer `setup.py` et `requirements.txt` dans les projets modernes.

BONNES PRATIQUES — QUALITÉ DE CODE ET GIT WORKFLOW

```
# Workflow Git feature branch
git checkout -b feature/gestion-utilisateurs
# ... développement ...
git add src/module.py tests/test_module.py
git commit -m "feat: ajouter la gestion des utilisateurs inactifs"
git push origin feature/gestion-utilisateurs
# Ouvrir une Pull Request → revue de code → merge

# Outils de qualité Python (installer une fois par projet)
pip install ruff black mypy pytest

ruff check .          # linter ultra-rapide (remplace flake8 + isort)
black .               # formatage automatique du code
mypy src/             # vérification statique des types
pytest tests/ -v      # lancer les tests unitaires avec détail
```

- Conventions de commit : Conventional Commits (feat: , fix: , docs: , refactor: , test:)
- Les hooks pre-commit automatisent les vérifications avant chaque commit

Configuration pre-commit minimale (*.pre-commit-config.yaml*):

```
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.0
    hooks:
      - id: ruff
      - id: ruff-format
```

Après `pre-commit install`, chaque `git commit` déclenche automatiquement ruff.

BONNES PRATIQUES — CI/CD AVEC GITHUB ACTIONS

```
# .github/workflows/ci.yml
name: CI Python

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - name: Configurer Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
      - name: Installer les dépendances
        run: pip install -r requirements.txt ruff pytest
      - name: Linter
        run: ruff check .
      - name: Tests
        run: pytest tests/ -v
```

CI/CD en deux mots : (tests automatiques à chaque commit) + (mise en production automatique si les tests passent). Ajoutez une étape de déploiement avec les actions de votre hébergeur : Railway, Fly.io, Heroku, AWS Elastic Beanstalk. Le badge de statut CI (`![CI](https://github.com/.../badge.svg)`) dans le README est un signal de qualité pour les collaborateurs.

RESSOURCES POUR ALLER PLUS LOIN

- Documentation officielle — <https://docs.python.org/fr/3> (disponible en français)
- PyPI — <https://pypi.org> — le dépôt officiel des paquets Python (+500 000 paquets)
- Real Python — <https://realpython.com> — tutoriels de qualité, gratuits et payants
- Exercism.io — <https://exercism.org/tracks/python> — exercices guidés avec feedback humain
- Kaggle — <https://www.kaggle.com> — compétitions ML, notebooks, jeux de données gratuits
- Awesome Python — <https://awesome-python.com> — liste curatée de bibliothèques par domaine
- Python Weekly — newsletter hebdomadaire sur l'actualité Python

Certifications Python reconnues : PCAP (Python Institute, niveau intermédiaire), PCPP (avancé). Pour la data science : Google Professional Data Engineer, AWS Machine Learning Specialty, Microsoft Azure Data Scientist Associate.

Communautés francophones : (Association Francophone Python — <https://www.afpy.org>), conférence annuelle, forum (section Python).

RÉCAPITULATIF DU MODULE & SUITE DU PARCOURS

- Data science : NumPy → pandas → Matplotlib/Seaborn → scikit-learn
- Web & APIs : Flask/FastAPI pour créer, `requests` pour consommer
- Bases de données : sqlite3 pour débiter, SQLAlchemy ORM pour les projets
- Cloud : boto3 (AWS), azure-sdk, google-cloud-python — même philosophie
- Qualité : venv + ruff + black + pytest + pre-commit + GitHub Actions
- Python est un langage **généraliste** — une seule syntaxe, de nombreuses portes ouvertes

Plan d'action recommandé pour la semaine suivante :

1. Installez Jupyter Lab et réexécutez les exemples du cours
2. Créez un petit projet personnel qui vous motive (bot Discord, tableau de bord, scraper, API)
3. Mettez ce projet sur GitHub avec un README et un workflow CI minimal
4. Rejoignez une communauté (Discord Python, Slack AFPy) pour ne pas rester seul face aux problèmes

“Le meilleur projet Python est celui que vous finirez vraiment.”